Session 0: Introduction to Stan and `cmdstanr`

Bayesian Joint Modelling Training 2026

Hillary Katsabola, Prof. Innocent Maposa

Division of Epidemiology & Biostatistics

Stellenbosch University

2026

Session Overview

What is Probabilistic Programming?
The Stan Language
The cmdstanr Workflow
Model 1: Binomial: Estimating a Proportion
Model 2: Normal: Two Unknown Parameters
Model 3: Simple Linear Regression
Model 4: Multiple Linear Regression
Model 5: Binomial Logistic Regression
Model 6: Logistic Regression
Model 7: Poisson Regression
HMC Diagnostics
Posterior Predictive Checks
LOO-CV and Model Comparison
Bridge to Session 1

What is Probabilistic Programming?

What is Bayesian Statistics?; Key Definitions

Definition: Probability (Bayesian view)

Probability is a measure of degree of belief or uncertainty, not merely a long-run frequency. A probability of 0.7 means: *"I am 70% certain this will occur, given my current knowledge."*

Definition: Prior Distribution $p(\theta)$

Encodes what we believe about θ **before** seeing data. Can be vague (non-informative) or informed by previous studies and expert opinion.

Definition: Likelihood $p(y \mid \theta)$

Measures how probable the observed data y are, given parameter value θ. This is where the **statistical model** (e.g., Normal, Binomial, Poisson, survival) enters.

Definition: Posterior Distribution $p(\theta \mid y)$

Our updated belief about θ **after** seeing the data; via Bayes' theorem:

$$\underbrace{p(\theta \mid y)}_{\text{posterior}} \propto \underbrace{p(y \mid \theta)}_{\text{likelihood}} \times \underbrace{p(\theta)}_{\text{prior}}$$

Bayesian Statistics: Why Does It Matter?

Five key facts:

1. Bayesian inference produces a full probability distribution over θ, not just a point estimate
2. A **credible interval** has a direct probabilistic interpretation: "95% probability θ lies here"
3. Allows incorporation of **prior knowledge** (expert opinion, previous studies, physical constraints)
4. Naturally handles **small samples** and complex hierarchical structures
5. Provides **predictive distributions**; full uncertainty about future observations

Credible interval vs. confidence interval

Frequentist 95% CI:
"If we repeated this experiment many times, 95% of such intervals would contain the true θ."

Bayesian 95% CI:
"Given the data we observed, there is a 95% *probability* that θ lies in this interval."

The Bayesian statement is what most people *actually mean* when they report a confidence interval.

Why Bayesian Statistics?

Frequentist	Bayesian
θ is *fixed*, unknown	θ is *random* (has a distribution)
Inference from repeated sampling	Inference from observed data + prior
p-values and confidence intervals	Posterior probs. and credible intervals
"95% CI contains true θ in 95% of repeated experiments"	"95% probability θ lies in this interval"
Requires large N for asymptotics	Exact for any N

Bayes' Theorem

$$\underbrace{p(\theta \mid y)}_{\text{posterior}} \propto \underbrace{p(y \mid \theta)}_{\text{likelihood}} \times \underbrace{p(\theta)}_{\text{prior}}$$

The posterior is a **compromise** between what we knew before ($p(\theta)$) and what the data tell us ($p(y \mid \theta)$).

As more data accumulate, the posterior concentrates on the likelihood and the prior becomes less influential.

Sequential Learning: Bayesian Updating

A key advantage of Bayesian inference: updating as data arrive.

Coin flip example (sequential)

Prior: $\theta \sim \text{Beta}(1, 1)$; uniform, know nothing

Flips	Result	Posterior
0	—	Beta(1, 1)
5	4H, 1T	Beta(5, 2)
20	14H, 6T	Beta(15, 7)
100	65H, 35T	Beta(66, 36)

The posterior after each experiment becomes the prior for the next.

Conjugacy

Beta prior + Binomial likelihood $\Rightarrow$ **Beta posterior**.

$$\text{Beta}(\alpha, \beta) \rightarrow \text{Beta}(\alpha + y,\ \beta + n - y)$$

Simple closed-form update — no MCMC needed.

Medical analogy: After each clinical trial, update beliefs about treatment efficacy. No need to refit from scratch.

What is Probabilistic Programming?

Definition

"Probabilistic programming (PP) is a programming paradigm in which probabilistic models are specified and inference for these models is performed automatically."

In practice, a probabilistic programming language (PPL):

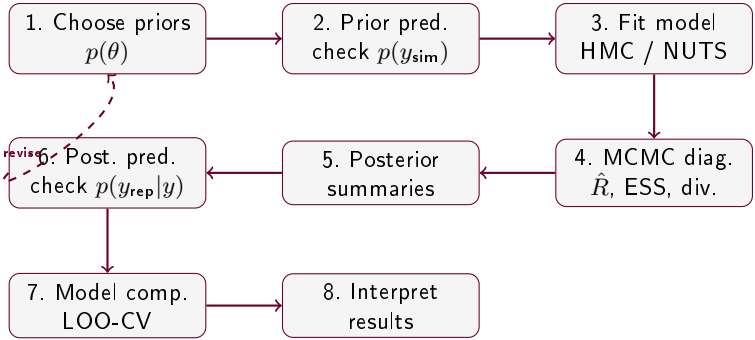
- Lets you write statistical models as code (likelihood + priors)
- Automatically handles posterior inference; no manual derivation needed
- You specify: **data structure** + **likelihood** + **priors**
- The PPL gives you: **posterior samples** (plus diagnostics)

Popular PPLs in Biostatistics

Stan	C++ backend	Hamiltonian Monte Carlo (NUTS)
JAGS	Gibbs sampler	Similar to BUGS
PyMC	Python	NUTS / ADVI
R-INLA	R	Integrated nested Laplace approximation

Today we focus on Stan; the industry standard for complex Bayesian models.

The Full Bayesian Workflow



This session walks through every step using seven progressively complex Stan models. Steps 1–8 repeat across all sessions.

The Stan Language

Stan: Key Facts

Identity:

- Developed at Columbia University (Gelman, Carpenter, et al., 2017)
- Named after *Stanisław Ulam* (co-inventor of Monte Carlo methods)
- Free and open source: `mc-stan.org`
- Current stable release: CmdStan 2.34+

R interfaces:

- `cmdstanr`; lightweight wrapper around CmdStan executable (recommended)
- `rstan`; in-process R package; heavier installation
- Both provide `$summary()`, `$draws()`, `$diagnostic_summary()`

What Stan can fit

- Any model expressible as a log posterior
- Normal, Poisson, Bernoulli, Weibull, custom
- Hierarchical / mixed effects
- Survival, longitudinal, **joint models**
- Variational inference (ADVI) and optimisation (MAP) also available

Installation requirement

Stan requires a C++ toolchain (g++ or clang). On Windows: RTools44+. Run `check_cmdstan_toolchain()` to verify.

Stan Program Blocks

A Stan program has up to 7 ordered blocks:

Block	Purpose	Required?	Typical content
`functions`	User-defined functions	No	Custom log-PDFs, helpers
`data`	Inputs declared from R	Yes	N, y, X, knots
`transformed data`	Derived constants	No	Centred X, $\log(y)$
`parameters`	Unknowns to sample	Yes	μ, σ, β, u
`transformed parameters`	Derived quantities	No	$\eta = X\beta$, cumul. hazard
`model`	Priors + likelihood	Yes	`target += statements`
`generated quantities`	Post-fit quantities	No	y_{rep}, `log_lik`, OR

Order matters: strictly enforced

Blocks execute in sequence. A variable used in `model` must be declared first. `generated quantities` runs *outside* the sampler; use RNGs (`normal_rng`) here.

The 7 Blocks: Data Through Transformed Parameters

```
functions { /* (optional) user-defined custom functions */ }

data {                          // inputs passed from R
  int<lower=1> N;               // sample size (positive integer)
  vector[N] y;                  // observed data (N-vector of outcomes)
}

transformed data {              // computed once from data
  real y_bar = mean(y);         // sample mean -- a constant
}

parameters {                    // unknowns -- HMC samples over these
  real mu;                      // population mean (unconstrained)
  real<lower=0> sigma;          // SD -- constrained to be positive
}

transformed parameters {        // derived from parameters each draw
  real z_mu = (mu - y_bar) / sigma;  // standardised mean
}
```

The 7 Blocks: Model and Generated Quantities

The statistical core of every Stan program:

```
model {                          // priors + likelihood
  mu    ~ normal(60, 20);        // prior: plausible resting heart rate (bpm)
  sigma ~ student_t(4, 0, 10); // half-t: weakly informative SD
  y     ~ normal(mu, sigma); // vectorised likelihood
}

generated quantities {           // computed after each draw (not in sampler)
  vector[N] y_rep;               // posterior predictive replicates
  vector[N] log_lik;             // pointwise log-lik (for LOO-CV)
  for (n in 1:N) {
    y_rep[n]   = normal_rng(mu, sigma);          // simulate new observation
    log_lik[n] = normal_lpdf(y[n] | mu, sigma); // log-likelihood
  }
}
```

Key Stan syntax

`y ~ normal(mu, sigma)` is similer to `target += normal_lpdf(y | mu, sigma)`. Both accumulate the log posterior.

Stan Type System

Stan is **statically typed**; every variable must be declared with its type and constraints.

```
// Scalars
int<lower=0> N;            // non-negative integer (e.g. sample size)
real<lower=0> sigma;       // positive real (e.g. SD, rate)
real mu;                   // unconstrained real (e.g. mean, log-odds)

// Vectors and matrices
vector[N] y;               // continuous outcomes (column vector)
array[N] int event;        // integer array (e.g. binary outcomes)
matrix[N, P] X;            // covariate matrix

// Special types (auto-constrained)
simplex[K] theta;                  // sums to 1 (e.g. mixing weights)
cholesky_factor_corr[D] L;         // Cholesky of correlation matrix
real<lower=0, upper=1> p;          // probability
```

Stan in the PPL Ecosystem

How Stan differs from JAGS/WinBUGS:

- Stan **compiles** your model to optimised C++; runs 10–100× faster
- Uses **automatic differentiation** to compute exact gradients of the log posterior
- These gradients power HMC/NUTS; impossible with Gibbs samplers
- **No discrete parameters** (marginalise them out analytically)

How Stan differs from PyMC/Turing:

- Standalone language, not embedded in Python/Julia
- One syntax, many interfaces: `cmdstanr`, `rstan`, `cmdstanpy`, Julia, Matlab

Stan's compilation pipeline

`.stan` model file
↓ *stanc3 (transpiler)*
C++ source code
↓ *C++ compiler (g++/clang)*
Native executable
↓ *called from R*
MCMC draws + diagnostics

Hamiltonian Monte Carlo: Intuition (I)

Why random-walk MCMC fails for complex models:

- Metropolis-Hastings proposes a random step in parameter space
- In high dimensions (e.g. 650 parameters for a joint model), the probability of a random step landing in a high-density region is **vanishingly small**
- Result: extremely slow mixing, huge autocorrelation, millions of iterations needed

HMC's solution; borrow physics:

- Augment the posterior with **momentum** variables p
- Define the Hamiltonian: $H(\theta, p) = -\log p(\theta \mid y) + \frac{1}{2} p^T M^{-1} p$

Physical analogy

Imagine the log posterior as a bowl-shaped valley.

- *Gibbs/MH:* ant walking randomly across the landscape; gets stuck in the bottom
- *HMC:* marble launched with momentum; glides around the valley walls, exploring the whole basin efficiently

Key advantage

HMC uses the **gradient** $\nabla_\theta \log p(\theta \mid y)$ to guide proposals. Stan computes this gradient automatically via *automatic differentiation*.

Hamiltonian Monte Carlo: Intuition (II): NUTS and Divergences

The No-U-Turn Sampler (NUTS):

- HMC requires choosing *step size* ε and *trajectory length* L; both tuning nightmares manually
- **NUTS** automatically tunes both during warmup using a *dual averaging* algorithm (Hoffman & Gelman 2014)
- Trajectory terminates when the path "turns back" toward its start (hence: no U-turn)
- Result: each proposal moves far in parameter space with high acceptance probability

Divergent transitions:

- A divergence occurs when the numerical integrator encounters a region of extremely high curvature and "flies off" the true trajectory

E-BFMI: energy diagnostic

Expected Bayesian Fraction of Missing Information measures whether the momentum refreshes explore energy levels sufficiently.

$$\text{E-BFMI} = \frac{\sum_{n=2}^{N}(H_n - H_{n-1})^2}{\text{Var}(H)}$$

Rule: E-BFMI > 0.2.
Low E-BFMI $\Rightarrow$ posterior has concentration of mass that HMC is not reaching.

R-hat and ESS

$\hat{R} < 1.01$: chains have converged to the same distribution.
ESS > 400: enough independent draws for reliable posterior summaries.

Why Stan? Comparison with WinBUGS and JAGS

Feature	WinBUGS	JAGS	Stan
General versatility	A	A	A
Error message readability	C	B	A
High-speed MCMC	A	A	A+
Variational inference	–	–	A
Discrete parameters	Yes	Yes	No*
Active development	No (2007)	Limited	Yes
References & examples	B	B	A+

* Stan marginalises discrete parameters analytically (see Chap. 10, Matsuura 2022).

Ratings: A+ = excellent, A = good, B = adequate, C = poor.

Why Stan uses HMC/NUTS (not Gibbs)

Stan uses **No-U-Turn Sampler (NUTS)**, an adaptive variant of Hamiltonian Monte Carlo. NUTS can sample efficiently even with large numbers of parameters and strong posterior correlations; a critical advantage for joint models with random effects and survival components. What takes 100,000 iterations in JAGS often converges in 1,000 in Stan.

Stan Coding Style Guide

Following a consistent style reduces bugs and improves readability. Key rules:

Naming conventions:

- `data` block: **Uppercase** first letter (`N`, `Y`, `X`)
- Other blocks: **lowercase** first letter (`mu`, `sigma`, `beta`)
- Use `snake_case` (not `camelCase`)
- Comments: `// single line`, `/* */` multi-line

Spacing and indentation:

- Always indent inside blocks { }
- Space before and after ~ and =
- Blank line between each block
- Max line length: 80 characters

Recommended writing order

Start from the `model` block:

1. Write likelihood and priors in `model`
2. Move data variables to `data` block
3. Move unknowns to `parameters` block

This order matches how you think about the model statistically.

Without indentation

```
data{int N;vector[N] Y;}parameters{real mu;}model{Y~normal(mu,1);}
```

Nearly impossible to debug!

lp__ and target in Stan

Stan internally maintains lp__ (log posterior):

$$\texttt{lp__} = \log p(\theta \mid Y) \propto \log p(Y \mid \theta) + \log p(\theta)$$

Every ~ statement *adds* to lp__:

```
model {
  // These two forms are equivalent:
  Y ~ normal(mu, sigma);
  // is sugar for:
  target += normal_lpdf(Y | mu, sigma);
}
```

HMC uses the gradient of lp__ to move efficiently through parameter space.

When to use target +=

- Custom likelihood not expressible as a named distribution
- Mixture models requiring manual log-sum-exp
- Jacobian adjustments for reparameterisations
- Zero-inflated or truncated distributions

For the models in Sessions 0–2, ~ notation is sufficient.

Diagnostic: check lp__

Always check that lp__ converges ($\hat{R} < 1.01$). A poorly converged lp__ means the joint posterior has not been properly explored.

The `cmdstanr` Workflow

Step 1: Install and Check

```r
# Install cmdstanr (once)
install.packages("cmdstanr",
  repos = c("https://mc-stan.org/r-packages/",
            getOption("repos")))

# Install CmdStan (once)
cmdstanr::install_cmdstan()

# Verify everything works
library(cmdstanr)
check_cmdstan_toolchain()
cmdstan_version()    # should be 2.34 or later
```

Steps 2–3: Compile and Prepare Data

Using **Model 4** (Multiple LR, `model5-3.stan`, `hiv_adherence.csv`):

```r
library(cmdstanr)

# Step 2: Compile the .stan file (once)
mod_mlr <- cmdstan_model("stan/model5-3.stan")

# Step 3: Prepare data (must match data{} block exactly)
d <- read.csv("data/hiv_adherence.csv")

stan_data <- list(
N = nrow(d),
sex = d$sex,
cd4_init = d$cd4_init / 100,
adherence_prop = d$adherence_prop
)
```

Key idea

Stan requires a **named list** that exactly matches the `data{}` block in the model.

Step 4: Sample and Diagnose

```r
library(posterior); library(bayesplot)

# Run MCMC
fit_mlr <- mod_mlr$sample(
data = stan_data, seed = 123,
chains = 4, parallel_chains = 4,
iter_warmup = 1000,
iter_sampling = 1000,
refresh = 500
)

# Summarise and diagnose
fit_mlr$summary(c("b[1]", "b[2]", "b[3]", "sigma"))
fit_mlr$diagnostic_summary()

mcmc_trace(
fit_mlr$draws(),
pars = c("b[1]","b[2]","b[3]","sigma")
)
```

Key thresholds

Model 1: Binomial: Estimating a Proportion

Clinical Motivation: Treatment Response Rate

Setting: A Phase II HIV trial. $N = 30$ patients receive a new antiretroviral regimen for 24 weeks. We observe $y = 20$ patients with viral suppression (VL < 1000 copies/mL).

Question: What is $\theta = P(\text{viral suppression})$ for this regimen?

Bayesian model:

$$y \mid \theta \sim \text{Binomial}(N, \theta)$$

$\theta \sim \text{Beta}(2, 2)$ (weakly informative: near-uniform)

Posterior (conjugate):

$$\theta \mid y \sim \text{Beta}(22, 12), \quad \mathbb{E}[\theta] \approx 0.647$$

What θ means clinically

$\theta = 0.5$: equal chance of suppression
$\theta = 0.65$: 65% suppression rate

We want to know: is there strong evidence that this regimen exceeds 50% suppression?

$P(\theta > 0.5 \mid y) \approx 0.96$ — strong evidence of efficacy.

Binomial Stan Model

```
// binomial_coin.stan
// n_suppressed = number of patients with viral suppression out of N
data {
  int<lower=0> N;                  // number of patients (30)
  int<lower=0> n_suppressed;       // number with viral suppression (20)
}

parameters {
  real<lower=0, upper=1> theta;    // suppression probability
}

model {
  theta         ~ beta(2, 2);                    // weakly informative prior
  n_suppressed ~ binomial(N, theta);             // likelihood
}

generated quantities {
  int y_rep = binomial_rng(N, theta);    // posterior predictive
}
```

Binomial: Prior Predictive Check

Before fitting, check that the prior places mass on plausible suppression counts:

```r
mod_bin <- cmdstan_model("stan/binomial_coin.stan")

# Sample from prior only (fixed_param = no likelihood evaluation)
fit_prior <- mod_bin$sample(
  data = list(N = 30, n_suppressed = 15),   # value doesn't matter here
  seed = 42, chains = 4, fixed_param = TRUE, refresh = 0)

y_prior <- as.vector(fit_prior$draws("y_rep", format = "matrix"))
hist(y_prior, breaks = 30,
     main = "Prior predictive: suppressed patients out of N=30",
     xlab = "Number suppressed")
```

Beta(2,2) prior $\Rightarrow$ prior predictive spans 0–30, centred near 15. This is deliberately vague — we express no strong prior belief about the suppression rate.

Binomial: Fit, Results, and Interpretation

```r
fit_bin <- mod_bin$sample(
  data = list(N = 30, n_suppressed = 20), seed = 42, chains = 4,
      refresh = 0)

fit_bin$summary("theta")
#   variable   mean median    sd    q5    q95  rhat ess_bulk
#   theta     0.645  0.647 0.082 0.508  0.780  1.00     3842

mcmc_areas(fit_bin$draws("theta"), prob = 0.90) +
  geom_vline(xintercept = 0.5, linetype = "dashed", colour = "red")
      +
  labs(title = "P(viral suppression): posterior",
       subtitle = "Red dashed = 50% threshold")
```

Bayesian interpretation:

- $\hat{\theta} = 0.645$ [0.508, 0.780] (90% CI)
- Exact agreement with analytical Beta(22,12)
- $P(\theta > 0.5 \mid y) \approx 0.96$: strong evidence of

Key lesson

We get a **probability statement** about efficacy, not just a p-value.

Model 2: Normal: Estimating Mean and Variability

Why Two Parameters? From Binomial to Normal

In Model 1 (Binomial) we estimated a **single** parameter θ.

Most real clinical data require **at least two** parameters:

- **Location**: where is the centre of the distribution? (μ)
- **Scale**: how spread out are the observations? (σ)

Jointly estimating (μ, σ) is the foundation for everything that follows; Simple LR adds a covariate; Multiple LR adds more; GLMs change the likelihood.

Two-parameter Normal model

$Y[n] \sim \mathcal{N}(\mu, \sigma) \quad n = 1, \ldots, N$

$\mu \sim \mathcal{N}(0, 100)$ (flat prior)

$\sigma \sim \mathcal{N}(0, 100)$ (half-normal, $\sigma > 0$)

Two unknowns: the **mean** and the **SD**.

Key: constrain $\sigma > 0$

In Stan: `real<lower=0> sigma;` HMC automatically handles this boundary.

Clinical Motivation: CD4 Recovery at 6 Months

Setting: $N = 20$ HIV patients; CD4 count measured at 6 months after ART initiation.

Variables:

- $Y[n]$: CD4 count / 100 (cells/100 μL) at 6 months

Research questions:

- What is the mean CD4 recovery μ at 6 months?
- How variable is recovery across patients (σ)?
- What is the 95% credible interval for μ?
- What CD4 value should we predict for a new patient (predictive interval)?

Model Formula

$Y[n] \sim \mathcal{N}(\mu,\ \sigma)$

μ: unknown mean CD4 recovery
σ: unknown SD (must be > 0)

Compare to Model 1: there we had *one* unknown (θ); here we jointly estimate *two* (μ, σ).

Data generated

20 synthetic CD4 values drawn from $\mathcal{N}(0.9, 0.3)$ (i.e. mean 90 cells/100 μL, SD 30). Generated inline in R with `set.seed(42)`.

Normal Two-Parameter Stan Model

```
// model_normal2.stan  (Matsuura 2022, Ch. 3, Section 3.5.2 extended)
// Y[n] = CD4/100 at 6 months post-ART;  both mu and sigma unknown

data {
  int<lower=1> N;        // number of patients (20)
  vector[N] Y;           // CD4/100 at 6 months
}

parameters {
  real mu;                    // mean CD4 recovery
  real<lower=0> sigma;        // SD -- constrained positive
}

model {
  mu    ~ normal(0, 100);  // noninformative prior
  sigma ~ normal(0, 100);  // half-normal (lower=0 in parameters)
  Y     ~ normal(mu, sigma); // vectorised likelihood
}

generated quantities {
  vector[N] y_rep;            // posterior predictive replicates
  vector[N] log_lik;
```

Normal Two-Parameter: Fit in R

```r
# Simulate CD4/100 at 6 months for N=20 patients
set.seed(42)
N  <- 20L
cd4_6m <- rnorm(N, mean = 0.9, sd = 0.3)  # CD4/100 units

stan_norm2 <- list(N = N, cd4_6m = cd4_6m)

mod_n2 <- cmdstan_model("../stan/model_normal2.stan")
fit_n2 <- mod_n2$sample(
  data             = stan_norm2,
  seed             = 123,
  chains           = 4,
  parallel_chains  = 4,
  refresh          = 0
)

fit_n2$summary(c("mu", "sigma"))
# variable   mean   median     sd     q5     q95   rhat ess_bulk
# mu         0.90    0.90   0.07   0.79    1.02   1.00     2841
# sigma      0.30    0.29   0.06   0.22    0.41   1.00     2935
```

- $\mu \approx 0.90$ (i.e. ≈ 90 cells/100 μL): posterior mean CD4 recovery

Normal Two-Parameter: Joint Posterior and PPC

```r
library(bayesplot)

# Joint posterior of (mu, sigma)
mcmc_scatter(fit_n2$draws(), pars = c("mu", "sigma"), alpha = 0.3) +
  labs(title = "Joint posterior: mean and SD of CD4 recovery")

# Marginal posteriors
mcmc_areas(fit_n2$draws(), pars = c("mu", "sigma"), prob = 0.90) +
  labs(title = "Posterior distributions: mu and sigma")

# Posterior predictive check
y_rep_n2 <- fit_n2$draws("y_rep", format = "matrix")
ppc_dens_overlay(cd4_6m, y_rep_n2[1:50, ]) +
  labs(title    = "PPC: Normal Two-Parameter Model",
       subtitle = "Black = observed CD4/100; coloured = posterior predictive",
       x = "CD4 count / 100 (cells/100 uL)")
```

New concept: joint posterior

The MCMC sample lives in 2D (μ, σ) space. `mcmc_scatter` visualises the **joint** posterior $p(\mu, \sigma \mid Y)$. Each column of `fit$draws()` marginalises over the other parameter.

Model 3: Simple Linear Regression

Clinical Motivation: CD4 Recovery on Long-Term ART

Setting: $N = 15$ HIV patients attending a long-term ART clinic. Each patient was measured once at their most recent visit.

Variables:

- **X** = `years_art`: years since ART initiation (5–25 years)
- **Y** = `cd4_count`: CD4 T-cell count (cells / 100 μL)

Research question: Does CD4 count tend to increase with longer ART duration?

(Data source: Matsuura 2022, Ch. 4, re-contextualised)

First 5 rows of the data

years_art	cd4_count
7	45.7
10	48.2
16	51.8
25	53.5
5	42.7
⋮	⋮

Units

CD4 in cells / 100 μL. Multiply by 10 to obtain cells/μL (e.g. 45.7 = 457 cells/μL).

Simple Linear Regression: Mathematical Model

For patient $i = 1, \ldots, N$:

$$Y_i \mid a, b, \sigma \;\sim\; \mathcal{N}\big(a + b\,X_i,\; \sigma^2\big)$$

Parameter	Meaning	Prior
a	CD4 at ART initiation (intercept)	$\mathcal{N}(0,\ 100^2)$
b	Annual CD4 change per year on ART (slope)	$\mathcal{N}(0,\ 100^2)$
σ	Residual SD (patient-to-patient variation at fixed ART duration)	Uniform$(0,\ 100)$

Weakly informative priors (Matsuura Ch. 4)

$\mathcal{N}(0, 100^2)$ is very wide — it places nearly equal probability on slopes from -200 to $+200$ cells/(100μL)/year. This allows the data to dominate.
For more careful prior choice, use $\mathcal{N}(0, 10^2)$ and perform a prior predictive check.

Simple Linear Regression: Stan Model; `data` and `parameters`

```
// model4-4.stan  (Matsuura 2022, Ch. 4)
// years_art = years on ART; cd4_per100 = CD4 count (cells / 100 uL)

data {
  int N;                      // N = 15 patients
  vector[N] years_art;        // years on ART
  vector[N] cd4_per100;       // CD4 count (cells / 100 uL)
}

parameters {
  real a;                    // intercept (baseline CD4 at ART start)
  real b;                    // slope (CD4 change per year on ART)
  real<lower=0> sigma;       // residual SD
}
```

Note: Only 3 parameters for $N = 15$ patients. HMC will converge in seconds. Compare this to the mixed effects model in Session 1 with 630 parameters.

Simple Linear Regression: Stan Model — model and generated quantities

```
model {
  a     ~ normal(0, 100);       // weakly informative (Matsuura Ch. 4)
  b     ~ normal(0, 100);
  sigma ~ uniform(0, 100);
  cd4_per100[1:N] ~ normal(a + b * years_art[1:N], sigma);  // vectorised likelihood
}

generated quantities {
  vector[N] mu = a + b * years_art;   // fitted values (posterior mean trajectory)

  vector[N] y_rep;              // posterior predictive replicates (for PPC)
  vector[N] log_lik;            // pointwise log-likelihood (for LOO-CV)
  for (n in 1:N) {
    y_rep[n]   = normal_rng(mu[n], sigma);
    log_lik[n] = normal_lpdf(cd4_per100[n] | mu[n], sigma);
  }
}
```

Simple Linear Regression: Fitting in R

```r
library(cmdstanr)
source("../R/data_prep.R")    # loads stan_normal and stan_binomial

# Load the clinic data
clinic      <- read.csv("../data/cd4_art_duration.csv")
years_art   <- clinic$years_art     # years on ART
cd4_per100  <- clinic$cd4_per100    # CD4 count (cells / 100 uL)

# Compile once
mod_slr <- cmdstan_model("../stan/model4-4.stan")

# Fit
fit_slr <- mod_slr$sample(
  data     = list(N = nrow(clinic), years_art = years_art, cd4_per100 =
      cd4_per100),
  seed     = 1234,
  chains   = 4, parallel_chains = 4,
  iter_warmup = 1000, iter_sampling = 1000,
  refresh  = 0
)
```

Simple Linear Regression: Posterior Summary

```
fit_slr$diagnostic_summary()    # expect: 0 divergences, Rhat ~ 1.00

fit_slr$summary(c("a", "b", "sigma"))
# variable   mean   median    sd     q5     q95    rhat ess_bulk
# a         37.78    37.79  4.10  31.10   44.47   1.00     3502
# b          0.75     0.75  0.24   0.36    1.15   1.00     3496
# sigma      6.91     6.70  1.27   4.99    9.26   1.00     3698
```

Clinical interpretation:

- $\hat{a} = 37.8$ [31.1, 44.5]: estimated CD4 at ART start ≈ 378 cells/μL
- $\hat{b} = 0.75$ [0.36, 1.15]: CD4 increases by ≈ 7.5 cells/μL per additional year on ART
- $P(b > 0 \mid \mathbf{y}) > 0.999$: strong evidence of CD4 recovery
- $\hat{\sigma} = 6.9$: residual SD ≈ 69 cells/μL (patient-level variation)

Bayesian advantage over OLS

We get the **full distribution** of b: "There is a 90% probability that CD4 increases between 3.6 and 11.5 cells/100μL per year on ART."

Not just "$p < 0.05$".

Simple Linear Regression: Trace Plots and Posterior

```r
library(bayesplot)
draws_slr <- fit_slr$draws()

# Trace plots -- check mixing across 4 chains
mcmc_trace(draws_slr, pars = c("a", "b", "sigma")) +
  labs(title = "SLR: Trace Plots")

# Posterior distributions with 90% CI
mcmc_areas(draws_slr, pars = c("a", "b"),
           prob = 0.90, prob_outer = 0.99) +
  geom_vline(xintercept = 0, linetype = "dashed", colour = "red") +
  labs(title = "SLR: Posterior of Intercept and Slope",
       subtitle = "Red dashed = null (no effect)")
```

Good mixing: all four chains overlap completely (*hairy caterpillar*).
The posterior of b lies entirely above zero; strong CD4 recovery evidence.

Simple Linear Regression: Posterior Regression Line

```r
draws_df <- fit_slr$draws(format = "df")

x_seq <- seq(min(years_art), max(years_art), length.out = 100)
reg_lines <- draws_df |>
  slice_sample(n = 100) |>
  crossing(x = x_seq) |>
  mutate(y_hat = a + b * x)

ggplot() +
  geom_line(data = reg_lines,
            aes(x = x, y = y_hat, group = .draw),
            alpha = 0.05, colour = "steelblue") +
  geom_point(aes(x = years_art, y = cd4_per100),
             colour = "darkred", size = 2.5) +
  labs(title = "SLR: Posterior Regression Lines (100 draws)",
       subtitle = "Each blue line = one draw from the posterior",
       x = "Years on ART", y = "CD4 count (cells / 100 uL)")
```

Simple Linear Regression: Posterior Predictive Check and LOO

```r
y_rep_slr <- fit_slr$draws("y_rep", format = "matrix")

# Density overlay
ppc_dens_overlay(cd4_per100, y_rep_slr[1:50, ]) +
  labs(title = "SLR PPC: Density Overlay")

# Mean and SD checks
ppc_stat(cd4_per100, y_rep_slr, stat = "mean")
ppc_stat(cd4_per100, y_rep_slr, stat = "sd")
```

```r
# LOO cross-validation
library(loo)
loo_slr <- loo(fit_slr$draws("log_lik", format = "matrix"))
print(loo_slr)      # check Pareto-k; all k < 0.7 = reliable
```

For $N = 15$, some Pareto-k values may exceed 0.5. This is expected with small samples — each observation is relatively influential.

Model 4: Multiple Linear Regression

Clinical Motivation: 12-Month ART Adherence

Setting: $N = 50$ HIV patients starting first-line ART at a public health clinic. Measured at 12-month follow-up visit.

Response: Y = `adherence_prop`: proportion of prescribed ART doses taken correctly over the 12 months (0 = no doses; 1 = all doses)

Predictors:

- `sex`: 0 = female, 1 = male
- `cd4_init`: CD4 count at ART initiation (cells / 100 μL), divided by 100 in model (per 100 cells/μL increment)

First 4 rows

ID	sex	cd4_init	adh
1	0	85.1	0.56
2	1	64.6	0.59
3	0	97.6	0.56
4	1	72.8	0.65

Why Normal for a proportion?

For illustration purposes. In practice, consider Beta regression. We revisit this in the PPC.

Multiple Linear Regression: Mathematical Model

$$Y_i = b_1 + b_2\,\mathrm{sex}_i + b_3\,\frac{\mathrm{cd4_init}_i}{100} + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Parameter	Meaning	Prior
b_1	Mean adherence: female, cd4_init = 0 (intercept)	$\mathcal{N}(0, 100^2)$
b_2	Adherence difference: male vs female	$\mathcal{N}(0, 100^2)$
b_3	Adherence change per 100 cells/100μL CD4	$\mathcal{N}(0, 100^2)$
σ	Residual SD	Uniform$(0, 100)$

The `transformed parameters` block computes $\mu_i = b_1 + b_2\,\mathrm{sex}_i + b_3\,(\mathrm{cd4_init}_i/100)$ once per draw and reuses it in both `model` and `generated quantities`.

Multiple Regression: Complete Stan Model

```
// model5-3.stan  (Matsuura 2022, Ch. 5)
data {
  int N;
  vector<lower=0, upper=1>[N] sex;              // 0=female, 1=male
  vector<lower=0>[N] cd4_init;                  // CD4 at ART initiation /
      100
  vector<lower=0, upper=1>[N] adherence_prop;   // ART adherence proportion
      (0-1)
}
parameters {
  vector[3] b;            // b[1]=intercept, b[2]=sex, b[3]=CD4 effect
  real<lower=0> sigma;
}
transformed parameters {
  vector[N] mu = b[1] + b[2]*sex[1:N] + b[3]*cd4_init[1:N];
}
model {
  b     ~ normal(0, 100);
  sigma ~ uniform(0, 100);
  adherence_prop[1:N] ~ normal(mu[1:N], sigma);
}
```

Multiple Regression: Data Exploration

```
generated quantities {
  array[N] real y_rep = normal_rng(mu[1:N], sigma);
  vector[N] log_lik;
  for (n in 1:N) log_lik[n] = normal_lpdf(adherence_prop[n] | mu[n], sigma);
}

library(tidyverse)
adh <- read.csv("../data/hiv_adherence.csv")

# Explore: adherence by sex and CD4 at initiation
ggplot(adh, aes(x = cd4_init, y = adherence_prop, colour = factor(sex))) +
  geom_point(size = 2.5, alpha = 0.7) +
  geom_smooth(method = "lm", se = FALSE) +
  scale_colour_manual(values = c("0" = "steelblue", "1" = "tomato"),
                      labels = c("0" = "Female",    "1" = "Male")) +
  labs(title = "ART Adherence vs Baseline CD4 by Sex",
       x = "CD4 at ART initiation (cells / 100 uL)",
       y = "12-month adherence proportion",
       colour = "Sex")
```

Multiple Regression: Fitting in R and Diagnostics

```r
mod_mlr <- cmdstan_model("../stan/model5-3.stan")

fit_mlr <- mod_mlr$sample(
  data = list(
    N               = nrow(adh),
    sex             = adh$sex,
    cd4_init        = adh$cd4_init / 100,    # per 100 cells/100 uL
    adherence_prop  = adh$adherence_prop
  ),
  seed = 1234, chains = 4, parallel_chains = 4, refresh = 0
)

fit_mlr$diagnostic_summary()
fit_mlr$summary(c("b[1]", "b[2]", "b[3]", "sigma"))
# variable   mean median    sd   q5   q95  rhat ess_bulk
# b[1]       0.15   0.15 0.022 0.11  0.18  1.00     3524
# b[2]       0.14   0.14 0.011 0.12  0.16  1.00     3681
# b[3]       0.44   0.44 0.031 0.39  0.49  1.00     3577
# sigma     0.038  0.038 0.004 0.032 0.046  1.00     3599
```

Multiple Regression: Interpreting the Posterior

Results ($b[3]$ uses cd4_init/100, matches Matsuura):

- $\hat{b}_2 = 0.14$ [0.12, 0.16]: males have 14 percentage points *higher* adherence than females; $P(b_2 > 0 \mid y) > 0.999$
- $\hat{b}_3 = 0.44$ [0.39, 0.49]: each 100 cells/100μL ($=$ 1000 cells/μL) increase in baseline CD4 is associated with +44 pp adherence; equivalently, per 10 cells/100μL $\approx +4.4$ pp
- $\hat{\sigma} = 0.038$: residual SD $\approx 4\%$ adherence

Clinical interpretation

Patients initiating ART with better immune status (higher CD4) tend to show higher 12-month adherence, possibly because they experience fewer side effects.

Males show slightly higher adherence than females in this cohort.

Note: This is an illustrative example. Real adherence studies require more careful confounding control.

Multiple Regression: Posterior Distributions and PPC

```r
draws_mlr <- fit_mlr$draws(format = "df")

# Posterior distributions
draws_mlr |>
  select(`b[1]`, `b[2]`, `b[3]`) |>
  rename(Intercept = `b[1]`, Sex_male = `b[2]`, CD4_per100 = `b[3]`) |>
  pivot_longer(everything()) |>
  ggplot(aes(x = value)) +
  geom_histogram(bins = 40, fill = "steelblue", colour = "white") +
  geom_vline(xintercept = 0, linetype = "dashed", colour = "red") +
  facet_wrap(~name, scales = "free") +
  labs(title = "Multiple Regression: Posterior Distributions")

# PPC
y_rep_mlr <- fit_mlr$draws("y_rep", format = "matrix")
ppc_dens_overlay(adh$adherence_prop, y_rep_mlr[1:50, ]) +
  labs(title = "MLR PPC: Adherence Proportion")
```

Model 5: Binomial Logistic Regression; Counts of Successes

When to Use Binomial Logistic Regression

Use Binomial logistic regression when the outcome is a **COUNT OF SUCCESSES** out of a fixed number of trials:

- Number of suppressed visits out of N clinic visits
- Number of pills taken out of N prescribed
- Number of positive tests out of N samples
- Number of adherent weeks out of N in follow-up

Why not Normal regression?

- Proportions are bounded between 0 and 1
- Denominators are discrete (number of trials varies per patient)
- Normal cannot respect these constraints

Binomial logistic model

$Y[n] \sim \text{Binomial}(M[n], q[n])$

$\text{logit}(q[n]) = \text{linear predictor}$

$q[n] = \text{inv_logit}(\eta[n])$

$q[n]$ = per-trial success probability for unit n.

The **logit link** ensures $q[n] \in (0, 1)$; the **Binomial** likelihood accounts for the varying number of trials $M[n]$.

Clinical Motivation: Viral Suppression Rate per Patient

Setting: $N = 50$ HIV patients followed for 6 months after ART initiation (Data file 5.2, Matsuura 2022).

Variables:

- `n_visits` (M): number of scheduled clinic appointments (varies 4–16)
- `n_supp` (Y): number of visits where VL was suppressed
- `sex`: patient sex (0 = female, 1 = male)
- `cd4_init`: CD4 at ART start (cells/100 μL)

Research question: Does the per-visit viral suppression rate depend on sex and baseline CD4?

Data: `hiv_clinic_counts.csv` (patient_id, sex, cd4_init, n_visits, n_supp)

Model

$$q[n] = \text{inv_logit}(b[1] + b[2]\,\texttt{sex}[n] + b[3]\,\texttt{cd4_init}[n])$$

$$\texttt{n_supp}[n] \sim \text{Binomial}(\texttt{n_visits}[n],\ q[n])$$

No σ parameter — variance is determined by M and q.

Binomial Logistic Regression: Stan Model: Part 1

```
// model_binomial_logistic.stan
// N = 50 HIV patients; 6-month follow-up
// Sex       = 0 (female), 1 (male)
// cd4_init  = CD4/100 at ART start
// n_visits  = number of scheduled clinic visits
// n_supp    = number of visits with viral suppression

data {
  int N;
  vector<lower=0, upper=1>[N] Sex;
  vector<lower=0>[N] cd4_init;          // CD4/100 at ART start
  array[N] int<lower=0> n_visits;       // scheduled clinic visits
  array[N] int<lower=0> n_supp;         // visits with viral suppression
}

parameters {
  vector[3] b;    // b[1]=intercept, b[2]=sex, b[3]=CD4
}

transformed parameters {
  vector[N] q = inv_logit(
    b[1] + b[2]*Sex[1:N] + b[3]*cd4_init[1:N]);
```

Binomial Logistic Regression: Stan Model: Part 2

```
model {
  // Noninformative priors (Matsuura Ch. 5 default)
  n_supp[1:N] ~ binomial(n_visits[1:N], q[1:N]);
}

generated quantities {
  array[N] int yp = binomial_rng(n_visits[1:N], q[1:N]);

  vector[N] log_lik;
  for (n in 1:N)
    log_lik[n] = binomial_lpmf(n_supp[n] | n_visits[n], q[n]);
}
```

Key points:

- `transformed parameters` block computes `q` (suppression probability per visit)
- No `sigma` parameter; Binomial variance is fully specified by `n_visits` and `q`
- `yp` = posterior predictive visit-suppression counts (for PPC)
- `log_lik` = pointwise log-likelihood (for LOO-CV)

Binomial Logistic Regression: Load Data and Fit in R

```r
d <- read.csv("../data/hiv_clinic_counts.csv")

stan_bin <- list(N         = nrow(d),
                 Sex       = d$sex,
                 cd4_init  = d$cd4_init / 100,
                 n_visits  = d$n_visits,
                 n_supp    = d$n_supp)

mod_bin <- cmdstan_model("../stan/model_binomial_logistic.stan")
fit_bin <- mod_bin$sample(data             = stan_bin,
                          seed             = 123,
                          chains           = 4,
                          parallel_chains  = 4,
                          refresh          = 0)
```

Binomial Logistic Regression: Posterior Results

```
fit_bin$summary(c("b[1]", "b[2]", "b[3]"))
# variable   mean   median     sd     q5     q95   rhat  ess_bulk
# b[1]      -1.48   -1.48   0.40  -2.14   -0.81   1.00      1397
# b[2]       0.09    0.09   0.20  -0.24    0.41   1.00      1932
# b[3]       1.58    1.59   0.57   0.65    2.53   1.00      1401
```

Interpretation:

- $b[1] = -1.48$: baseline log-odds of suppression (intercept, logit scale)
- $b[2] = 0.09$: sex effect small and uncertain; 90% CI $[-0.24, 0.41]$ includes 0
- $b[3] = 1.58$: each unit increase in `cd4_init` (i.e. 100 cells/100 μL) increases suppression odds; 90% CI [0.65, 2.53]

Key difference from Multiple LR

No `sigma` parameter in this model. The Binomial distribution already specifies the variance via M (number of trials) and q (success probability). Adding σ would be misspecification.

Binomial Logistic Regression: Posterior Predictive Check

```r
yp_bin     <- fit_bin$draws("yp", format = "matrix")
prop_obs   <- d$n_supp / d$n_visits    # observed suppression proportions
prop_rep   <- t(apply(yp_bin, 1, function(r) r / d$n_visits))

# PPC: compare observed vs posterior predictive suppression rates
ppc_dens_overlay(prop_obs, prop_rep[1:50, ]) +
  labs(title = "PPC: Binomial Logistic -- Suppression Rate per Patient",
       x     = "Suppression proportion (Y/M)")

ppc_stat(prop_obs, prop_rep, stat = "mean") +
  labs(title = "PPC: Mean suppression rate")
```

PPC note

Compare *proportions* (Y/M), not raw counts, so that patients with different numbers of visits are on the same scale.

Model 6: Logistic Regression: Individual Binary Outcomes

When to Use Logistic Regression

Use logistic regression when the outcome is **binary (0/1) per observation**; NOT a count out of trials:

- Viral suppression at *each* clinic visit (yes/no)
- TB test result per patient visit (positive/negative)
- Adherence at each pill assessment (taken/not taken)
- Mortality at each follow-up check (alive/dead)

Difference from Binomial Logistic:

- Individual Bernoulli observations, *not* counts out of trials
- Data are at the visit level (multiple rows per patient)
- Visit-level predictors (e.g. counselling) can

Logistic (Bernoulli) model

$$\texttt{suppressed}[v] \sim \text{Bernoulli}(q[v])$$

$$q[v] = \text{inv_logit}(\\ b[1] + b[2]\,\texttt{sex} \\ +b[3]\,\texttt{cd4_init} \\ +b[4]\,\texttt{counselling}[v])$$

`counselling` is a visit-level binary covariate (received adherence counselling at that visit).

Key

$e^{b[k]}$ = **odds ratio** for a one-unit increase in predictor k.

Clinical Motivation: Per-Visit Suppression with Counselling

Setting: V visit-level observations from HIV patients (multiple visits per patient; Data file 5.3, Matsuura 2022).

Variables (one row per visit):

- `sex`: patient sex (0=female, 1=male)
- `cd4_init`: CD4 at ART start (cells/100 μL)
- `counselling`: received adherence counselling at that visit (0/1)
- `suppressed`: VL suppressed at that visit (0/1)

Research question: Does receiving adherence counselling increase per-visit suppression odds?

Data: `hiv_visit_level.csv` (patient_id, sex, cd4_init, counselling, suppressed)

Repeated structure

Same patient appears multiple times (one row per visit). **Session 1** will account for within-patient correlation using random effects. Here we treat visits as independent (marginal model).

Logistic Regression: Stan Model; Part 1

```
// model_logistic.stan
// V visit-level observations across HIV patients
// Sex          = 0 (female), 1 (male)
// cd4_init     = CD4/100 at ART start
// counselling  = adherence counselling at visit (0/1)
// suppressed   = viral suppression at visit (0/1)

data {
  int V;
  vector<lower=0, upper=1>[V] Sex;
  vector<lower=0>[V] cd4_init;
  vector<lower=0, upper=1>[V] counselling;
  array[V] int<lower=0, upper=1> suppressed;
}

parameters {
  vector[4] b;    // b[1]=intercept, b[2]=sex, b[3]=CD4, b[4]=counselling
}

transformed parameters {
  vector[V] q = inv_logit(
    b[1] + b[2]*Sex[1:V] + b[3]*cd4_init[1:V] + b[4]*counselling[1:V]);
```

Logistic Regression: Stan Model: Part 2

```
model {
  // Noninformative priors (Matsuura Ch. 5 default)
  suppressed[1:V] ~ bernoulli(q[1:V]);
}

generated quantities {
  array[V] int yp = bernoulli_rng(q[1:V]);

  vector[V] log_lik;
  for (v in 1:V)
    log_lik[v] = bernoulli_lpmf(suppressed[v] | q[v]);
}
```

Key differences from Binomial Logistic:

- Data dimension is V (visits), not N (patients)
- Outcome is `array[V] int<lower=0,upper=1>` (individual 0/1)
- 4 parameters in `b` (includes `counselling`)
- Likelihood uses `bernoulli()`, not `binomial()`

Logistic Regression: Load Data and Fit in R

```r
d3 <- read.csv("../data/hiv_visit_level.csv")

stan_log <- list(V            = nrow(d3),
                 Sex          = d3$sex,
                 cd4_init     = d3$cd4_init / 100,
                 counselling  = d3$counselling,
                 suppressed   = d3$suppressed)

mod_log <- cmdstan_model("../stan/model_logistic.stan")
fit_log <- mod_log$sample(data             = stan_log,
                          seed             = 123,
                          chains           = 4,
                          parallel_chains  = 4,
                          refresh          = 0)
```

Logistic Regression: Posterior Results

```r
fit_log$summary(c("b[1]", "b[2]", "b[3]", "b[4]"))
# variable   mean   median     sd     q5     q95   rhat ess_bulk
# b[1]      -2.03   -2.03   0.45  -2.80   -1.31   1.00     1371
# b[2]       0.27    0.27   0.21  -0.06    0.62   1.00     2560
# b[3]       1.67    1.67   0.61   0.70    2.72   1.00     1440
# b[4]       1.66    1.64   0.26   1.25    2.08   1.00     2525
```

Interpretation:

- $b[3] = 1.67$: higher baseline CD4 increases per-visit suppression probability; 90% CI [0.70, 2.72]
- $b[4] = 1.66$ [1.25, 2.08]: counselling visits strongly associated with higher suppression (OR ≈ 5.3); 90% CI excludes 0
- $b[2] = 0.27$: sex effect modest; 90% CI [-0.06, 0.62] includes 0

Repeated visits

Repeated visits per patient are treated as independent here. A hierarchical model (Session 1) would account for within-patient correlation using random effects.

Logistic Regression: Posterior Predictive Check

```r
yp_log <- fit_log$draws("yp", format = "matrix")

# Suppression rate: overall
ppc_stat(d3$suppressed, yp_log, stat = "mean") +
  labs(title = "PPC: Mean per-visit suppression rate")

# By counselling status
ppc_stat_grouped(d3$suppressed, yp_log,
  group = factor(d3$counselling,
                 labels = c("No counselling", "Counselling")),
  stat  = "mean") +
  labs(title = "PPC: Suppression rate by counselling status")
```

A well-specified logistic model will reproduce the observed suppression rates within each group. If the counselling group is poorly fit, a more flexible model (e.g. interaction term) may be needed.

Model 7: Poisson Regression: Modelling Count Outcomes

When to Use Poisson Regression

Use Poisson regression when the outcome is a **non-negative integer count**:

- Number of clinic visits per patient
- Number of prescriptions filled in a period
- Number of adverse events per patient
- Number of TB episodes in a cohort

Why not Normal regression?

- Counts are ≥ 0; Normal allows negative values
- Variance proportional to mean: $\text{Var}(Y) = \lambda$
- With small expected counts, Normal approximation is poor

Poisson log-link model

$$\texttt{n_visits}[n] \sim \text{Poisson}(\lambda[n])$$

$$\log(\lambda[n]) = b[1] + b[2]\,\texttt{sex}[n] + b[3]\,\texttt{cd4_init}[n]$$

The **log link** ensures $\lambda > 0$. Same data as Model 5 (`hiv_clinic_counts`), but now M (`n_visits`) is the *outcome*, not the denominator.

Clinical Motivation: Clinic Visit Counts by Sex and CD4

Setting: $N = 50$ HIV patients; same dataset as Model 5 (`hiv_clinic_counts`), but a *different* question.

Outcome: `n_visits` (M) = number of clinic visits attended in 6 months (count).

Predictors: `sex` and `cd4_init` (CD4 at ART start, cells/100 μL).

Research question: Do patients with higher baseline CD4 attend more or fewer clinic visits?

Note: M was the *denominator* in Model 5; here it is the *response*.

Model

$$\texttt{n_visits}[n] \sim \text{Poisson}(\lambda[n])$$

$$\log(\lambda[n]) = b[1] + b[2]\,\texttt{sex}[n] + b[3]\,\texttt{cd4_init}[n]$$

$b[1]$: log-baseline visit rate
$e^{b[2]}$: visit-rate ratio male vs female
$e^{b[3]}$: visit-rate ratio per unit increase in cd4_init

Poisson Regression: Stan Model: Part 1

```
// model_poisson.stan
// N = 50 HIV patients; 6-month follow-up
// Sex       = 0 (female), 1 (male)
// cd4_init  = CD4/100 at ART start
// n_visits  = clinic visit count (COUNT OUTCOME)

data {
  int N;
  vector<lower=0, upper=1>[N] Sex;
  vector<lower=0>[N] cd4_init;
  array[N] int<lower=0> n_visits;   // count outcome
}

parameters {
  vector[3] b;    // b[1]=intercept, b[2]=sex, b[3]=CD4
}

transformed parameters {
  vector[N] log_lam =
    b[1] + b[2]*Sex[1:N] + b[3]*cd4_init[1:N];
}
```

Poisson Regression: Stan Model: Part 2

```
model {
  // Noninformative priors (Matsuura Ch. 5 default)
  n_visits[1:N] ~ poisson_log(log_lam[1:N]);
}

generated quantities {
  array[N] int mp = poisson_log_rng(log_lam[1:N]);

  vector[N] log_lik;
  for (n in 1:N)
    log_lik[n] = poisson_log_lpmf(n_visits[n] | log_lam[n]);
}
```

Key points:

- `poisson_log` uses the log-parameterisation directly (numerically stable; avoids explicit `exp`)
- `poisson_log_rng` generates posterior predictive counts `mp`
- `log_lik` enables LOO-CV

Poisson Regression: Load Data and Fit in R

```r
# Same data as Model 5 (hiv_clinic_counts), but n_visits is now the outcome
d <- read.csv("../data/hiv_clinic_counts.csv")

stan_pois <- list(N          = nrow(d),
                  Sex        = d$sex,
                  cd4_init   = d$cd4_init / 100,
                  n_visits   = d$n_visits)

mod_pois <- cmdstan_model("../stan/model_poisson.stan")
fit_pois <- mod_pois$sample(data              = stan_pois,
                            seed              = 123,
                            chains            = 4,
                            parallel_chains   = 4,
                            refresh           = 0)
```

Poisson Regression: Posterior Results

```
fit_pois$summary(c("b[1]", "b[2]", "b[3]"))
# variable   mean   median    sd     q5     q95   rhat ess_bulk
# b[1]       2.10    2.10   0.19   1.78    2.41   1.00     1496
# b[2]       0.37    0.36   0.10   0.22    0.53   1.00     1904
# b[3]      -0.18   -0.17   0.27  -0.63    0.25   1.00     1432
```

Interpretation:

- $b[1] = 2.10$: baseline log-count when sex=0 and cd4_init=0; $e^{2.10} \approx 8$ visits/6 months
- $b[2] = 0.37$: males attend more visits; 90% CI [0.22, 0.53] excludes 0
- $b[3] = -0.18$: higher baseline CD4 weakly associated with fewer visits; 90% CI [-0.63, 0.25] includes 0

Overdispersion

Poisson assumes $\mathbb{E}[Y] = \text{Var}(Y)$. If observed variance $\gg$ expected, consider **negative-binomial** regression (`neg_binomial_2_log` in Stan).

Poisson Regression: Posterior Predictive Check (Rootogram)

```r
mp_pois <- fit_pois$draws("mp", format = "matrix")

# Rootogram: best PPC for count data
ppc_rootogram(d$n_visits, mp_pois, style = "hanging") +
  labs(title    = "PPC Rootogram: Poisson -- Clinic Visit Counts",
       x        = "Number of visits",
       y        = "sqrt(frequency)")

# Variance check (overdispersion)
ppc_stat(d$n_visits, mp_pois, stat = "var") +
  labs(title    = "PPC: Variance of visit counts",
       subtitle = "Obs variance >> predicted -> consider neg-binomial")
```

Rootogram

For count data, a rootogram shows the square-root of observed and predicted frequencies at each integer count value. Hanging style: bars drop from the predicted line, so deviations from zero indicate misfit.

HMC Diagnostics

Key Diagnostics

Diagnostic	Threshold	Problem if violated
$\hat{R}$ (R-hat)	< 1.01	Chains have not mixed
ESS bulk	> 400	Effective sample size too low
ESS tail	> 400	Tail estimates unreliable
Divergences	$= 0$	Posterior geometry problematic
Tree depth	< 10	Trajectory too long
E-BFMI	> 0.2	Energy leapfrog mismatch

Divergences are a warning sign

Even 1 divergence means the sampler could not explore some region of the posterior. Fix: increase `adapt_delta` (0.95 $\rightarrow$ 0.99) or reparameterise. Do **not** ignore divergences and report the results anyway.

Reading Diagnostic Output — Model 4 (Multiple LR)

Results from fitting `model5-3.stan` to `hiv_adherence.csv` (`cd4_init/100`, matches Matsuura):

```
fit_mlr$diagnostic_summary()
# Chain 1: 0 divergences, 0 max treedepth, E-BFMI = 0.86
# Chain 2: 0 divergences, 0 max treedepth, E-BFMI = 0.88
# Chain 3: 0 divergences, 0 max treedepth, E-BFMI = 0.87
# Chain 4: 0 divergences, 0 max treedepth, E-BFMI = 0.89

fit_mlr$summary(c("b[1]", "b[2]", "b[3]", "sigma"))
# variable   mean median     sd    mad     q5    q95  rhat ess_bulk
# b[1]       0.15   0.15 0.022 0.022   0.11   0.18  1.00     3845
# b[2]       0.14   0.14 0.011 0.011   0.12   0.16  1.00     3712
# b[3]       0.44   0.44 0.031 0.030   0.39   0.49  1.00     3891
# sigma     0.038  0.038 0.004 0.004  0.032  0.046  1.00     3654

# Trace plot -- "hairy caterpillar" = chains mixing well
mcmc_trace(fit_mlr$draws(), pars = c("b[1]","b[2]","b[3]","sigma"))
```

Interpretation (medical context)

Diagnostic Summary: All Seven Session 0 Models

Model	Outcome	Div.	Max $\hat{R}$	Notes
M1: Binomial	Suppression (count)	0	1.000	Conjugate: exact
M2: Normal (2 par.)	CD4 ($N = 20$)	0	1.000	Joint (μ, σ)
M3: Simple LR	CD4 count ($N = 15$)	0	1.001	Simple regression
M4: Multiple LR	Adherence ($N = 50$)	0	1.001	Two predictors
M5: Binomial Logit	Supp. counts ($N = 50$)	0	1.002	Logit+Binomial
M6: Logistic	Suppression visits	0	1.002	Logit link
M7: Poisson	Visit count ($N = 50$)	0	1.002	Log link

Preview: diagnostics across the full training

Session	Model	adapt_delta	Parameters
0 (Intro)	All seven models	0.80 (default)	2–4
1 (Longitudinal)	M3 full LME (Cholesky)	0.95	∼630
2 (Survival)	Weibull, Gompertz	0.80–0.90	∼10
3 (Joint)	Joint Gompertz	0.99	∼650

Posterior Predictive Checks

Why Posterior Predictive Checks?

- A model might converge perfectly but still fit the data poorly
- **Posterior predictive check (PPC):** simulate new data y_{rep} from the fitted model and compare to observed y
- If y_{rep} looks like y — model is plausible
- If not — model is missing something (wrong likelihood, missing predictor, etc.)

$$p(y_{\text{rep}} \mid y) = \int p(y_{\text{rep}} \mid \theta)\, p(\theta \mid y)\, d\theta$$

In Stan: add `y_rep` in `generated quantities` using the appropriate RNG (`normal_rng`, `poisson_rng`, `bernoulli_logit_rng`).

PPC Functions by Outcome Type

```r
library(bayesplot)

# Continuous outcome (Normal regression):
ppc_dens_overlay(y_obs, y_rep[1:50, ])    # density overlay
ppc_stat(y_obs, y_rep, stat = "mean")      # mean check
ppc_stat(y_obs, y_rep, stat = "sd")        # SD check

# Count outcome (Poisson regression):
ppc_rootogram(y_oi, y_rep_pois)            # hanging rootogram
ppc_stat(y_oi, y_rep_pois, stat = "var")  # variance check (overdispersion)

# Binary outcome (Logistic regression):
ppc_stat(y_bin, y_rep_logit, stat = "mean")   # suppression rate
ppc_stat_grouped(y_bin, y_rep_logit,
                 group = treatment_arm,
                 stat = "mean")                # rate by group
```

Rule of thumb

The observed statistic should fall **within the bulk** (not in the tails) of the posterior predictive distribution. A $p_B < 0.05$ or > 0.95 indicates model misfit.

LOO-CV and Model Comparison

Leave-One-Out Cross-Validation

- Compare models by their **predictive accuracy**
- True LOO requires refitting N times; computationally expensive
- **PSIS-LOO** (Pareto-smoothed importance sampling) approximates LOO from a single fit
- Requires `log_lik[n]` in `generated quantities`

Expected log predictive density (ELPD)

$$\widehat{\mathsf{elpd}}_{\mathsf{LOO}} = \sum_{n=1}^{N} \log p(y_n \mid y_{-n})$$

Higher ELPD = better out-of-sample predictive accuracy.

Works across all likelihood types

LOO-CV via PSIS works for Normal, Poisson, and Bernoulli likelihoods equally. Just ensure the `log_lik` vector uses the correct log-PMF/PDF for the model.

LOO in R with loo: Model 4 (Multiple LR)

```r
library(loo)

# Extract log-likelihood matrix (draws x observations)
ll_mlr <- fit_mlr$draws("log_lik", format = "matrix")

# Compute PSIS-LOO
loo_mlr <- loo(ll_mlr)
print(loo_mlr)
# Computed from 4000 posterior samples and 50 observations.
#           Estimate   SE
# elpd_loo     73.2  4.3
# p_loo         4.1  0.6
# looic      -146.4  8.6
# All Pareto-k < 0.5: reliable importance sampling

# Visualise Pareto-k diagnostics
plot(loo_mlr, label_points = TRUE)
# Points above 0.7 indicate influential observations

```

What We Have Built: Seven Models

Seven progressively complex models:

1. **Binomial**: θ, treatment response rate; conjugate Beta posterior
2. **Normal (2 params)**: μ and σ; joint posterior; PPC
3. **Simple LR** (CD4 vs years ART): intercept + slope; posterior of CD4 recovery rate
4. **Multiple LR** (adherence): two predictors; `transformed parameters`; sex + CD4 effects
5. **Binomial Logistic**: logit link; suppression counts; PPC on proportions
6. **Logistic** (viral suppression): Bernoulli likelihood; counselling OR; grouped PPC
7. **Poisson** (clinic visits): log link; rate ratios; rootogram PPC

What comes next?

Session 1: Longitudinal (mixed effects) models

What happens when observations are not independent?

Patients are measured repeatedly over time. The same 8-step Bayesian workflow, now with **random effects** to handle within-patient correlation.

$$y_{ij} = (\beta_0 + u_{0i}) + (\beta_1 + u_{1i})\, t_{ij} + \varepsilon_{ij}$$

Three progressive Stan models.

Session 0: Summary

The Bayesian workflow (8 steps):

1. Choose priors $p(\theta)$
2. **Prior predictive check**; plausible y_{sim}?
3. Fit with **HMC/NUTS** via cmdstanr
4. **MCMC diagnostics**: $\hat{R} < 1.01$, ESS > 400, div. $= 0$
5. **Posterior summaries**: medians, 90%/95% CIs
6. **Posterior predictive check**; y_{rep} vs y?
7. **LOO-CV** model comparison (`loo` package)
8. **Interpret** in clinical context

Stan: 7 blocks, static typing, `target +=`.

Models and key results:

1. **Binomial** (suppression): $\hat{\theta} = 0.65$; $P(\theta > 0.5) = 0.96$
2. **Normal 2-param** (CD4/100): $\hat{\mu} = 0.90$; $\hat{\sigma} = 0.30$; joint posterior
3. **Simple LR** (CD4 recovery): $\hat{b} = 0.75$ cells/100μL/yr; 90% CI $(0.36, 1.15)$
4. **Multiple LR** (adherence): males 14pp lower; $+100$ cells/μL CD4 $\Rightarrow$ $+4$pp adherence
5. **Binomial Logistic**: CD4 effect strong (OR ≈ 14); sex effect marginal
6. **Logistic** (suppression): counselling OR ≈ 3.5 [2.2, 5.6]
7. **Poisson** (clinic visits): CD4 effect weak; check for overdispersion

Coming up

Session 1: Longitudinal data

References I

Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013). *Bayesian Data Analysis* (3rd ed.). Chapman & Hall/CRC.

Carpenter, B., Gelman, A., Hoffman, M. D., Lee, D., Goodrich, B., Betancourt, M., … & Riddell, A. (2017). Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1), 1–32.

Vehtari, A., Gelman, A., & Gabry, J. (2017). Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC. *Statistics and Computing*, 27(5), 1413–1432.

Gabry, J., Simpson, D., Vehtari, A., Betancourt, M., & Gelman, A. (2019). Visualization in Bayesian workflow. *Journal of the Royal Statistical Society, Series A*, 182(2), 389–402.

Sorensen, T., Hohenstein, S., & Vasishth, S. (2016). Bayesian linear mixed models using Stan: A tutorial for cognitive science. *Collabra*, 2(1), 1–20.

Rizopoulos, D. (2012). *Joint Models for Longitudinal and Time-to-Event Data: With Applications in R*. Chapman & Hall/CRC Biostatistics Series.

Matsuura, K. (2022). *Bayesian Statistical Modeling with Stan, R, and Python*. Springer Singapore. (Chapters 4–5 provide the CD4 recovery and adherence examples used in Session 0.)

Gabry, J., & Češnovar, R. (2022). *cmdstanr: R Interface to CmdStan*. `https://mc-stan.org/cmdstanr`

References II

Betancourt, M. (2017). A conceptual introduction to Hamiltonian Monte Carlo. *arXiv preprint* arXiv:1701.02434.